Book-Writer: An Autonomous Multi-Agent Pipeline for Overnight Long-Form Book Generation with Locally Hosted Language Models

LI JAR · SuperJAR
Orchast Agent Project
August 2026

***Abstract*—**

Generating a complete book with a large language model is constrained by context length, unattended-operation fragility, and the cost of iterating against metered cloud APIs. We present Book-Writer, an autonomous system that turns a table of contents into a complete, typeset, version-controlled book overnight using only open-weight models served locally by Ollama. The system decomposes a book into chapters and processes each through a four-stage sequential pipeline of specialized agents—outliner, writer, reviewer, and finalizer—whose stages communicate through typed session state. An imperative orchestrator provides the machinery unattended operation requires: a fresh model session per chapter, per-chapter retry and timeout, graceful degradation to the best available stage output, disk-persisted resumable progress, and an incremental version-control commit of every completed chapter. A deterministic, model-free publisher stage then typesets the chapters into versioned PDF books without a TeX toolchain, and the results are distributed through a public web library. Over roughly three months of routine operation the system produced 16 complete books—258 chapters, 536,295 words, and 1,886 PDF pages—in English, Korean, and Burmese, at a median wall-clock cost of 7.1 minutes per chapter and zero inference cost. We describe the architecture and its rationale, report an observational evaluation over the public corpus, and discuss the limitations of scale-oriented evaluation of machine-written books.

***Index Terms*—multi-agent systems, large language models, long-form text generation, local inference, autonomous agents, publishing pipeline**

I. Introduction

Large language models produce fluent prose at paragraph and article scale, yet book-scale generation remains difficult in practice. A complete book of several hundred pages exceeds the effective context of most models, so a single generation pass cannot maintain structure from the first chapter to the last. Quality is equally hard to sustain: a first draft produced in one shot receives no editorial attention, and errors accumulate silently across tens of thousands of words. Finally, the economics of iteration matter. Producing, discarding, and regenerating hundreds of chapters against a metered cloud API is expensive enough to discourage exactly the experimentation that long-form generation requires.

Prior work addresses parts of this problem. Recursive prompting and revision schemes such as Re3 [1] extend story length beyond a single context window, and outline-driven systems such as STORM [2] ground article-scale writing in a research-then-write process. General multi-agent frameworks, including AutoGen [3] and MetaGPT [4], demonstrate that decomposing a task across specialized model roles improves output quality. However, these systems target stories or articles rather than complete typeset books, typically assume frontier cloud models, and give little attention to the systems engineering required for multi-hour unattended operation: crash recovery, partial-progress persistence, and publication of the result as a distributable artifact.

This paper presents Book-Writer, an autonomous system that writes complete books overnight using only locally hosted open-weight models served by Ollama [5]. The system decomposes a book into chapters and processes each chapter through a four-stage sequential pipeline of specialized agents—outliner, writer, reviewer, and finalizer—built on the Google Agent Development Kit (ADK) [6]. An imperative orchestrator wraps the pipeline with the machinery that unattended operation demands: a fresh model session per chapter to bound context growth, per-chapter retry and timeout, disk-persisted progress with resume, and a version-controlled commit of every chapter as it completes. A deterministic publisher stage then typesets the accumulated chapters into a versioned PDF, and the results are distributed through a public web library. In roughly three months of operation the system produced 16 complete books comprising 258 chapters, 536,295 words, and 1,886 PDF pages across three

languages, at a median cost of 7.1 minutes of wall-clock time per chapter and zero inference cost.

The contributions of this work are:

- a chapter-level decomposition of book generation into a four-stage generate-and-refine agent pipeline whose stages communicate through typed session state;
- an orchestration design for unattended multi-hour runs on local models, combining per-chapter sessions, retry with graceful degradation, resumable progress, and incremental version-controlled publication;
- a deterministic publishing stage that turns pipeline output into typeset, versioned PDF books without a TeX toolchain; and
- an observational evaluation over a public corpus of 16 machine-written books in English, Korean, and Burmese.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture and its design rationale. Section IV details the implementation. Section V reports the evaluation over the generated corpus. Section VI discusses findings and limitations, and Section VII concludes.

II. Related Work

A. Long-Form Text Generation

Extending coherent generation beyond a single context window is a recognized challenge. Re3 [1] generates long stories through recursive reprompting and revision, alternating drafting with editing passes—an approach the present work echoes in its writer–reviewer–finalizer chain, though at chapter rather than passage granularity. STORM [2] shows that separating outline construction from article writing improves the structure of Wikipedia-like articles; Book-Writer applies the same outline-first discipline to every chapter. Both systems, however, target single documents of article or story length and rely on hosted frontier models, whereas Book-Writer produces multi-hundred-page typeset books on open-weight models running on local hardware.

B. Multi-Agent LLM Frameworks

AutoGen [3] and MetaGPT [4] established that assigning specialized roles to cooperating model instances yields better results than a single generalist prompt, particularly when one role reviews another's output. These frameworks emphasize flexible conversation topologies. Book-Writer deliberately adopts the simplest possible topology—a fixed sequential chain—on the grounds that book chapters need a repeatable production line rather than negotiation. The system is built on the Google Agent Development Kit [6], which supplies the agent abstraction, sequential composition, and session-state passing, while model access is routed through LiteLLM [7] to Ollama [5], which serves open-weight models such as the Gemma family [8] on local hardware.

C. Position of This Work

The gap Book-Writer occupies is the combination of scale, autonomy, and cost: book-length output assembled from chapter-level pipeline runs, produced unattended overnight, on hardware the operator already owns. To our knowledge, publicly documented systems that publish complete multilingual, typeset, version-controlled books from local models are rare, and the engineering required for that autonomy—recovery, resumption, and incremental publication—is the least-reported part of comparable systems.

III. System Architecture

A. Design Overview

Book-Writer treats the chapter, not the book, as the unit of generation. The input is a table of contents (TOC) declaring the book's title, description, and an ordered list of chapters, each with a title and a short description. An orchestrator iterates over the chapters and runs each one through a four-stage pipeline of specialized language-model agents; completed chapters are written to disk and committed to version control immediately. When all chapters exist, a deterministic publisher stage typesets them into a versioned PDF. Fig. 1 shows the complete flow.

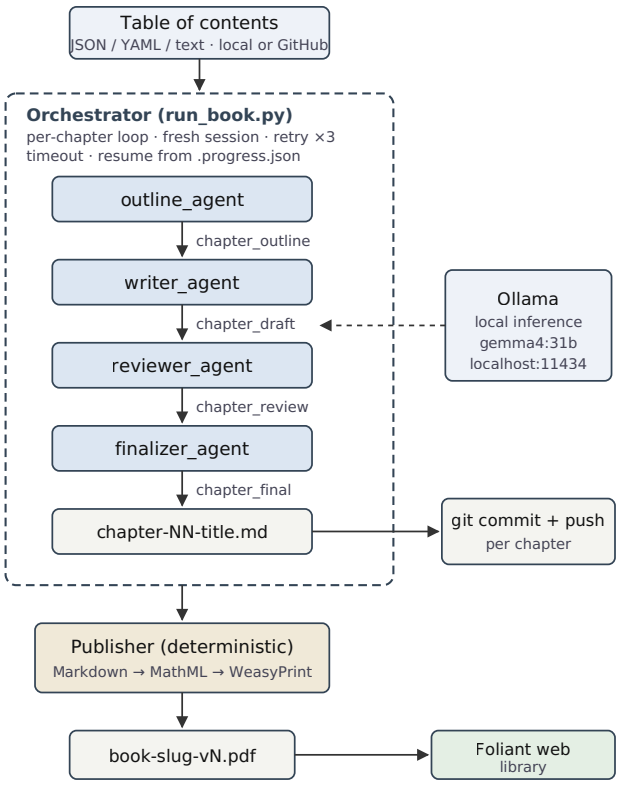

Fig. 1. Book-Writer architecture. The orchestrator runs the four-stage agent pipeline once per chapter with a fresh session, commits each completed chapter to version control, and finally invokes the deterministic publisher, whose PDFs are distributed through the Foliant web library.

B. Input Model and Operating Modes

The TOC is deliberately the system's entire authoring interface. It is accepted as JSON or YAML, or as plain numbered text for the lowest-friction case, and may carry optional fields that shape generation without touching code: a target language, per-book writing guidelines injected into the generation prompts, and per-chapter descriptions that seed each outline. The TOC may also be given as a URL into a hosted repository, in which case the orchestrator clones the repository and derives the output location and branch from the TOC's position within it—so a book "lives" in its own repository from the first chapter.

The pipeline's stages are individually selectable at invocation time, which turns one program into several tools. Running only the publisher re-typesets an existing book without any model invocation; omitting the reviewer and finalizer produces fast draft-quality output for inspecting a TOC's viability; regeneration flags rewrite selected chapters, or the whole book, in place. The same mechanism separates concerns during development, since any stage's behavior can be exercised in isolation against real intermediate artifacts.

C. The Chapter Pipeline

Each chapter passes through four agents composed as a strict sequence. Writing s_i for the specification of chapter i taken from the TOC, the produced chapter is $c_i = f_{fin}(f_{rev}(f_{wr}(f_{out}(s_i))))$, where the four functions are the outline, writer, reviewer, and finalizer agents. The stages communicate through named session state: each agent writes its full output under a designated key, and the next agent's prompt template interpolates that key. Table I lists the stages and their contracts.

TABLE I — Pipeline stages and their state contracts.

Stage	Role	Reads	Writes
outline	plans 3–6 sections, key points, word budget	TOC entry	`chapter_outline`
writer	full prose draft following the outline	`chapter_outline`	`chapter_draft`
reviewer	rewrites for clarity, flow, completeness	outline + draft	`chapter_review`
finalizer	production polish, formatting discipline	`chapter_review`	`chapter_final`
publisher	typesets all chapters to PDF (no LLM)	chapter files	versioned PDF

Two design choices in the pipeline are deliberate. First, the reviewer is a rewriting editor, not a critic: its instruction requires it to output the complete improved chapter rather than review notes, which removes a merge-feedback step that weaker local models handle unreliably. Second, every stage is forbidden from emitting meta-commentary, so any stage's output is a usable chapter; this property underpins the orchestrator's degradation strategy described next.

D. Orchestration for Unattended Operation

The orchestrator is an imperative loop rather than a framework-level loop construct, because the requirements of an overnight run sit outside what declarative composition offered: chapter-specific state must be injected into each run,

progress must survive process death, and failures must be contained to a single chapter. Each chapter executes in a brand-new in-memory session, so context never accumulates across chapters and the memory footprint stays bounded regardless of book length. A failed or timed-out chapter is retried up to three times with a fresh session; if every attempt fails, the failure is recorded and the loop simply proceeds to the next chapter. When a run is interrupted, a progress file records completed, failed, and in-flight chapters, and a resume flag skips work that already exists on disk.

The orchestrator also degrades gracefully within a chapter. Because every stage outputs a complete chapter, the runner can fall back through the state keys in order—final, review, draft, outline—and persist the best available text if the tail of the pipeline fails. Finally, each completed chapter is committed and pushed to the output repository immediately, with pull–rebase retries on push conflicts; a night's work is therefore preserved chapter-by-chapter even if the machine dies before morning.

E. The Publisher as a Deterministic Stage

The fifth stage contains no language model at all. Typesetting is a deterministic transformation, and making it one keeps content generation and presentation independently re-runnable: a book can be re-typeset without touching a model, and chapters can be regenerated without re-typesetting until the end. The publisher's output is versioned automatically, so successive publications of the same book coexist as v1, v2, and so on.

IV. Implementation

A. Stack and Footprint

Book-Writer is implemented in 1,388 lines of Python across four modules: the agent definitions (196 lines), the tool library including the publisher (486 lines), the command-line orchestrator (671 lines), and a small FastAPI wrapper (35 lines) for interactive use. Agents are ADK `Agent` instances composed with `SequentialAgent` [6]; model access goes through LiteLLM [7] to an Ollama server [5] on the local host. The default model is `gemma4:31b` from the Gemma open-weight family [8], with generation configured at temperature 0.7, a 32,768-token context window, and a repetition penalty of 1.2. Three command-line flags—disabling model thinking, shrinking the context window, and raising the repetition penalty—adapt the pipeline to models small enough to run on a Raspberry Pi, the same class of machine that hosts the orchestrator itself.

B. Prompt Design

Each stage's instruction follows a common pattern: a one-sentence professional role ("You are a book production editor performing the final polish"), the chapter's coordinates within the book, the upstream stage's output interpolated from session state, and an explicit output contract. The contracts do most of the work. The writer must produce "substantive prose paragraphs, NOT bullet points," must begin with a canonical chapter heading, and must emit "ONLY the chapter content in Markdown. No meta-commentary." The finalizer enforces seven mechanical checks, including heading hierarchy, no orphaned headings, and the absence of leftover review notes or TODO markers. User-supplied writing guidelines from the TOC are injected into the outline, writer, and reviewer prompts, and a language directive supporting 23 languages forces non-English books to be written entirely in the target language while permitting code and technical terms to remain in English.

C. Robustness Machinery

A striking property of the implementation is how much of it exists to keep an unattended run alive rather than to generate text. Table II summarizes the mechanisms; together they account for the bulk of the orchestrator's 671 lines. Before any generation begins, the runner verifies that Ollama is reachable and the requested model is available.

TABLE II — Robustness mechanisms for unattended operation.

Mechanism	**Behavior**
Health check	verify Ollama endpoint and model before starting
Per-chapter retry	up to 3 attempts, fresh session each attempt
Timeout	1,800 s per chapter attempt via async cancellation
Fallback capture	persist best stage output if late stages fail
Progress file	completed/failed/in-progress state on disk; resume flag
Idempotent restart	chapters already on disk are skipped
Incremental git	commit and push after every chapter; 3 pull–rebase retries
Failure isolation	a chapter that exhausts retries is logged, loop continues

D. Publishing and Distribution

The publisher reads the chapter files in order, strips their metadata front matter, converts Markdown to HTML, translates embedded LaTeX math to MathML, and renders an A4 PDF with WeasyPrint [9]—a CSS-based typesetting path that avoids a TeX toolchain entirely. The generated document includes a title page and a table of contents with page references resolved by the layout engine, and output files are versioned automatically. Finished chapters and PDFs accumulate in a public repository [10], and a companion web application, Foliant [11], presents the collection as a browsable library with a public book-request page, closing the loop from a TOC file to a distributed, readable book.

V. Evaluation

A. Method

The evaluation is observational: it measures what the system actually produced during roughly three months of routine operation, rather than a controlled benchmark. All measurements were taken on 24 August 2026 from a snapshot of the public output repository [10]. Chapter and word counts were computed from the chapter source files with metadata front matter removed; PDF page counts were extracted programmatically from the latest published version of each book; and per-chapter generation times were derived from the generation timestamps embedded in consecutive chapters' front matter, discarding gaps longer than three hours as idle time between sessions. Timing figures are therefore estimates of wall-clock production time, not instrumented measurements.

B. Corpus Scale

Between 19 May and 12 August 2026 the system produced 16 complete books: 258 chapters totaling 536,295 words, published as PDFs totaling 1,886 pages. Eight books are in English, five in Korean, and three in Burmese—the multilingual output exercising the language directive rather than separate models. Books range from 7 to 29 chapters and from roughly 10,600 to 94,800 words; the largest single volume is a 29-chapter, 264-page treatise. Table III lists a representative subset spanning the size and language range.

TABLE III — Representative books from the generated corpus.

Book	Lang.	Ch.	Words	PDF pages
Atomic Theory of Society…	EN	29	94,770	264
Quantum Computing System Development	KO	25	47,488	177
C Programming (introductory)	MY	23	40,586	203
The Ethics of Cruelty	EN	21	48,856	144
The Wave Is Already Here	EN	15	47,070	136
Brain–Computer Interface Technologies	EN	14	39,311	131
The Weight of Light	EN	12	24,984	71
Ethical Hacking for Developers	EN	8	21,144	68
Hansel and Gretel (retelling)	KO	7	10,577	42
Corpus total (16 books)	—	**258**	**536,295**	**1,886**

C. Throughput

Across 239 usable consecutive-chapter intervals, the median production time was 7.1 minutes per chapter and the mean 15.1 minutes, with individual chapters ranging from 2.1 to 171 minutes. The gap between median and mean reflects a heavy right tail rather than uniform slowness: most chapters complete in single-digit minutes, while a minority run long—behavior consistent with the retry mechanism, which restarts a stalled chapter from scratch up to three times and therefore multiplies the wall-clock time of exactly the chapters that misbehave. This distribution is itself an argument for the per-chapter failure isolation described in Section III: a slow or failing chapter delays only itself.

At the median rate, a 20-chapter book completes in under three hours of generation time, and indeed every book in the corpus finished its chapters within a single day—consistent with the intended overnight usage pattern, in which the operator supplies a TOC in the evening and finds a committed, typeset book in the morning. Because inference runs on local hardware, the marginal cost of this output was electricity.

D. Durability in Practice

The corpus also evidences the robustness machinery working as designed. The output repository's history shows one commit per chapter followed by a publication commit per book, so every night's partial progress was preserved

incrementally. One title, *The Weight of Light*, exists in eight published PDF versions, demonstrating regeneration and re-publication of the same book over time; the automatic version numbering kept all editions addressable. The publisher stage handled all three scripts in the corpus—Latin, Hangul, and Burmese—producing paginated PDFs from the same pipeline without script-specific configuration.

VI. Discussion and Limitations

A. What the Design Buys

Three observations stand out from operating the system. First, chapter-level decomposition converts an intractable long-context problem into a sequence of tractable medium-context problems: no stage ever needs more context than one chapter's worth of material, which is why 30,000-token-class local models suffice for 90,000-word books. The cost of this decomposition is that no agent ever sees the whole book, a trade-off examined below. Second, the orchestrator—at 671 of the system's 1,388 lines, its largest module—is dominated by robustness machinery rather than generation logic. For unattended operation this inversion appears essential: over a multi-hour run against a local inference server, timeouts, stalls, and push conflicts are routine events, and the corpus was produced through them, not in their absence. Third, keeping typesetting deterministic and model-free made presentation iterable at zero model cost; the eight published versions of one book were re-typeset and re-published without regenerating text.

B. Limitations

The evaluation measures scale, throughput, and durability—not literary or technical quality. No human evaluation, readability scoring, or factuality audit of the generated books has been performed, and for the non-fiction titles the risk of confidently stated model error is real; the books are published with an explicit machine-authorship disclaimer. Cross-chapter consistency is structurally unenforced: because each chapter is generated in a fresh session, nothing prevents terminology drift or contradiction between chapters beyond what the shared TOC implies.

The implementation carries known weaknesses. There is no automated test suite. Configuration passes from orchestrator to agents through environment variables read at import time, an ordering-sensitive coupling, and the stage-name registry is duplicated between two modules that must be kept in sync manually. The finalizer's prompt omits the user's writing guidelines that the three earlier stages receive—an inconsistency rather than a design decision. The math converter falls back silently to literal text on conversion failure, so malformed formulas can reach published PDFs unnoticed. The Ollama endpoint is hardcoded to the local host, and chapters are generated strictly sequentially, leaving multi-machine or parallel generation unexplored. Finally, the reported timing figures inherit the precision of file timestamps rather than instrumentation.

C. Future Work

The natural next steps target the two structural gaps: a lightweight cross-chapter memory (for example, a running glossary and character or concept sheet injected into each chapter's session) to address consistency, and an automated quality pass—readability metrics, factual spot-checks, and sampled human review—to make the quality of the corpus measurable. On the engineering side, a test suite around the TOC parser, progress tracking, and publisher, plus parallel chapter generation across multiple Ollama hosts, would extend the same design without altering it.

VII. Conclusion

This paper described Book-Writer, an autonomous system that turns a table of contents into a complete, typeset, version-controlled book overnight using only locally hosted open-weight models. Its contribution is less any single component than the demonstrated combination: a four-stage generate-and-refine agent pipeline scoped to the chapter, an orchestrator engineered for unattended multi-hour operation, and a deterministic publishing stage—which together produced a public corpus of 16 books, 258 chapters, and 536,295 words across English, Korean, and Burmese at a median of 7.1 minutes per chapter and no inference cost. The system's operating history suggests that for long-form generation, the binding constraints are no longer model access or expense but consistency across generated units and the measurement of output quality; both are tractable extensions of the architecture presented here.

References

1. K. Yang, Y. Tian, N. Peng, and D. Klein, "Re3: Generating longer stories with recursive reprompting and revision," in *Proc. EMNLP*, 2022.
2. Y. Shao, Y. Jiang, T. Kanell, P. Xu, O. Khattab, and M. Lam, "Assisting in writing Wikipedia-like articles from scratch with large language models," in *Proc. NAACL*, 2024.

3. Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
4. S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. ICLR*, 2024.
5. Ollama, "Ollama: Run large language models locally," https://ollama.com/, 2025.
6. Google, "Agent Development Kit (ADK) for Python," https://github.com/google/adk-python, 2025.
7. BerriAI, "LiteLLM: Unified interface for LLM providers," https://github.com/BerriAI/litellm, 2025.
8. Google DeepMind, "Gemma open models," https://ai.google.dev/gemma, 2025.
9. CourtBouillon, "WeasyPrint: The awesome document factory," https://weasyprint.org/, 2025.
10. prof-lijar, "ai-generated-books: Books written overnight by AI agents," https://github.com/prof-lijar/ai-generated-books, 2026.
11. Foliant, "Foliant — a library of AI-written books," https://floriant-press.vercel.app/, 2026.